Contents

BigQuery

BigQuery is Google Cloud's serverless data warehouse. You write SQL, it stores data in columnar form and scans it across many parallel workers, and you pay for the bytes scanned. Storage and compute are decoupled, so a large table costs the same to keep whether you query it daily or once a year.

Generic questions

1. What is BigQuery?

BigQuery is Google Cloud's **serverless, fully managed data warehouse** for analytics. You write SQL; it stores data in **columnar** form on Colossus and runs queries on the **Dremel** engine across many parallel workers. **Storage and compute are decoupled**, so you store cheaply and scale compute on demand, and you pay for **bytes scanned** (on-demand) or reserved **slots** (capacity). It scales to petabytes with no servers to manage.

2. What problem does BigQuery solve?

Running **analytical queries over huge datasets fast, without managing infrastructure**. A traditional warehouse makes you size servers, build indexes, and tune storage; BigQuery removes all of that. Because storage and compute are separate, a large table costs the same to keep whether you query it often or never, and a heavy query borrows thousands of workers for a few seconds. It turns "provision and tune a cluster" into "write SQL".

3. Why would you choose BigQuery over a traditional relational database?

For **analytics at scale**. Columnar storage plus massively parallel scans beat a row store for aggregations over billions of rows, it is **serverless** (no provisioning or tuning), and storage and compute scale independently to petabytes.

Trap: this holds only for OLAP work. For single-row reads and writes, a relational database wins. BigQuery has seconds-level latency and limited mutations, so it is the wrong choice for a transactional backend.

4. Is BigQuery an OLTP or OLAP database?

OLAP. It is built for large scans and aggregations, not fast single-row reads and writes.

Aspect	OLTP (e.g. Postgres)	OLAP (BigQuery)
Workload	Many small reads and writes	Large scans and aggregations
Latency	Milliseconds	Hundreds of ms to seconds
Storage	Row-oriented	Columnar
Mutations	Cheap and frequent	Limited and costly

> **INTERVIEW TRAP**
>
> Never use BigQuery as an application's transactional backend. It is an analytics store; the app should write to its own operational database and let data flow into BigQuery for reporting.

5. What are the primary use cases of BigQuery?

- **Analytics and BI**: dashboards, reporting, and ad-hoc exploration over large data.
- **Data warehousing and ELT**: a central store many pipelines load into and query.
- **Event and log analytics**: append-heavy data queried by time.
- **Joining large datasets** for aggregation.
- **BigQuery ML**: train and run models in SQL.

The common thread is read-heavy, analytical work over large volume.

6. What are the limitations of BigQuery?

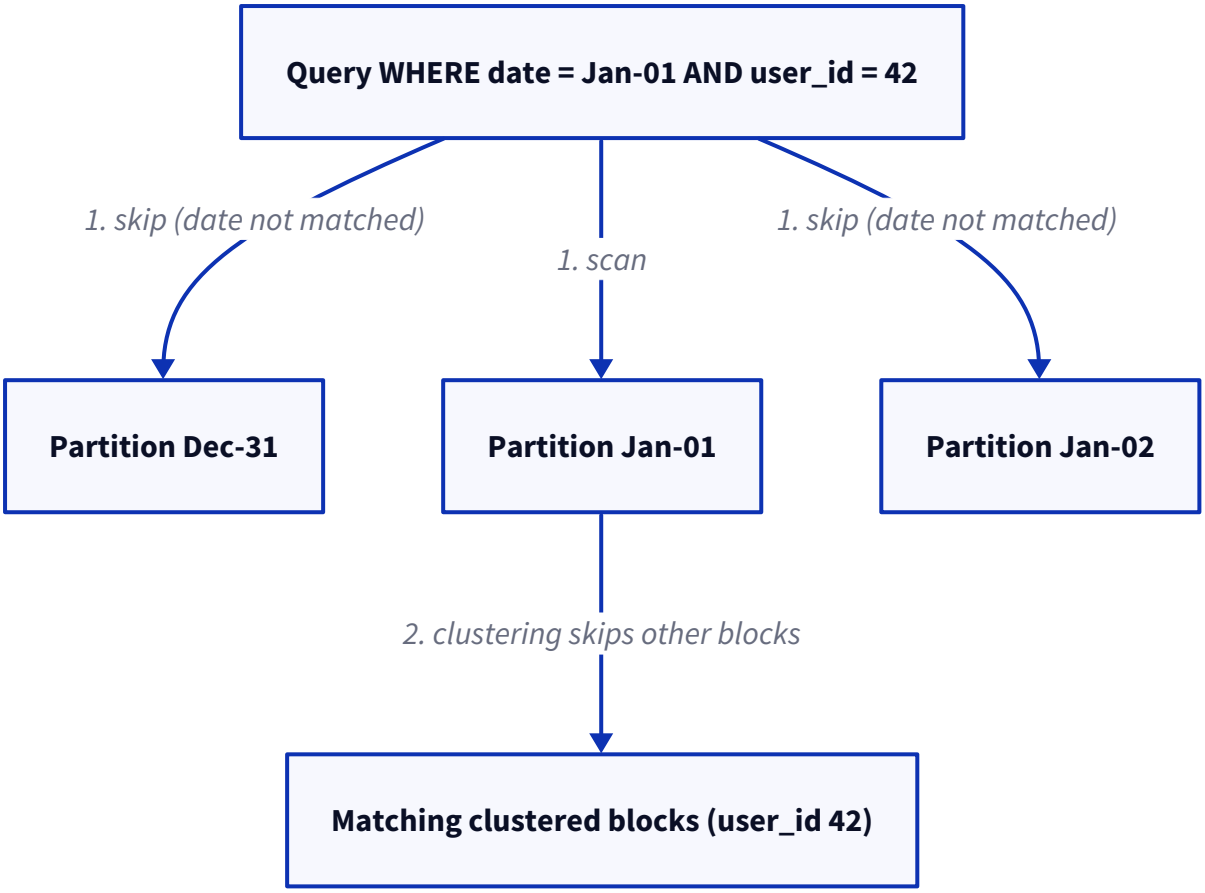
- **Not OLTP**: no sub-millisecond lookups, and query latency is seconds, not milliseconds.
- **Mutations are limited and costly**: frequent small updates and deletes do not fit; it is append-first.
- **No traditional indexes**, and **no enforced constraints** (primary and foreign keys are not enforced).
- **Cost surprises**: on-demand billing is per byte scanned, so a careless `SELECT *` on a big table is expensive.
- **Streaming** adds cost and a short window before the data is fully settled.

7. What is partitioning and clustering?

Both cut how much data a query reads, which is exactly what you pay for.

Partitioning splits a table into segments by a column, usually a **date or timestamp**. A query that filters on that column scans only the matching partitions, called **partition pruning**.

Clustering sorts the data **within** each partition by up to four columns. A query that filters or groups by those columns reads only the matching blocks, called **block pruning**.



Aspect	Partitioning	Clustering
Granularity	Coarse, whole partitions	Fine, blocks within a partition
Keys	One column (date or integer range)	Up to 4 columns
Best for	The time column every query filters	High-cardinality filter, join, or group keys

Rule of thumb: partition on the date you always filter by, cluster on the id you filter or join by next.

8. How do you debug slow queries?

Read the **query execution plan**. It shows each stage with records in and out, **slot time**, **shuffle bytes**, and time split across wait, read, compute, and write. Look for a stage that reads far more than expected (no pruning), heavy **shuffle** or a skewed stage where one worker does most of the work, or long **wait** time (slot contention). `INFORMATION_SCHEMA.JOBS` gives slot time and bytes per query. Common fixes: add a partition filter, cluster on the join key, drop `SELECT *`, and filter before joining.

9. How do you identify expensive queries?

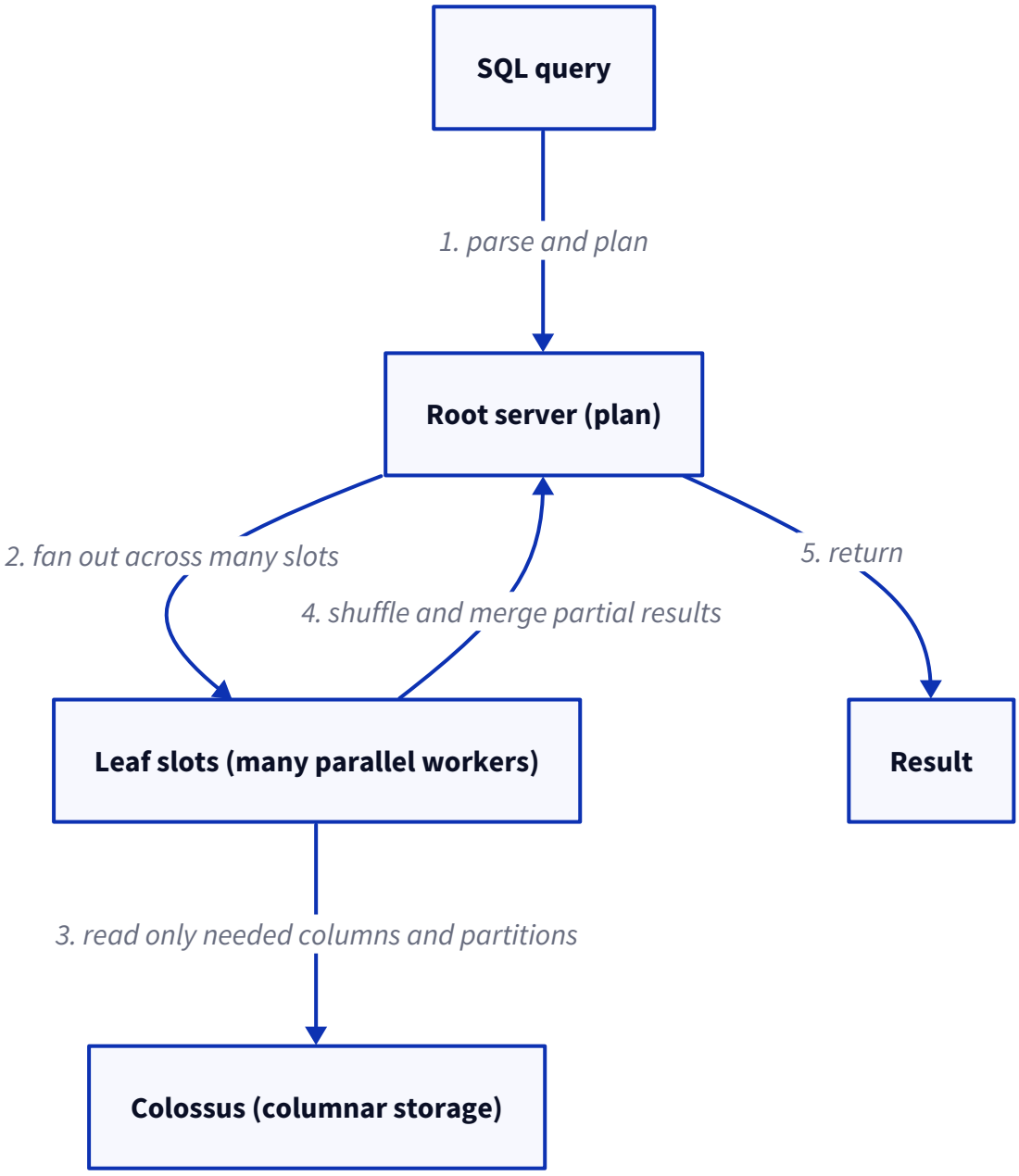
Cost tracks **bytes scanned**, so expensive means scans-a-lot. Before running, a **dry run** estimates the bytes a query will read. After the fact, query `INFORMATION_SCHEMA.JOBS` and rank by `total_bytes_billed` or `total_slot_ms` to find the worst offenders by user, project, or query. The billing export gives the same view for dashboards.

INTERVIEW TRAP

`SELECT *` bills for every column, because storage is columnar and each column is read separately. Select only the columns you need.

10. If BigQuery has no indexes, why are queries still fast?

Because it **scans in parallel instead of seeking**. Three things make that fast: **columnar storage** reads only the columns you select, not whole rows; **massive parallelism** fans the scan across thousands of slots reading from Colossus at once; and **pruning** from partitioning and clustering skips data the filter cannot match, doing the job an index would. So instead of one index seek, it runs a very wide parallel scan of only the needed columns and partitions.



11. Describe a production system where you used BigQuery.

An analytics pipeline for product event data. Services emitted events that were loaded into BigQuery, into a **date-partitioned, clustered** table, either streamed in near real time or batch-loaded on a schedule. That table powered dashboards, scheduled reports, and per-entity performance stats the product read back. BigQuery

was the analytics store, not the transactional one: the app wrote to its own operational database, and events flowed into BigQuery for reporting.

12. What was the largest dataset you have worked with?

A multi-terabyte event table, on the order of billions of rows and growing daily. It was **partitioned by day** and **clustered by an entity id**, so although the table was large, a typical query filtered to a day or two and one entity and scanned only a tiny fraction of it. The size never became a problem, because queries almost never touched the whole table.

13. You have a 20 TB table. How would you optimize queries?

Cut what each query reads:

- **Partition** on the time column and **require a partition filter**, so every query prunes by date.
- **Cluster** on the high-cardinality columns queries filter, join, or group by.
- **Select only the needed columns**, never `SELECT *`.
- **Pre-aggregate** repeated rollups with materialized views or scheduled tables.
- Use **approximate functions** like `APPROX_COUNT_DISTINCT` where exactness is not required.
- For dashboards, put **BI Engine** in front for an in-memory cache.

14. Have you optimized expensive BigQuery queries? What changes did you make?

Yes, and the biggest wins came from reading less data. I replaced `SELECT *` with the handful of columns actually used, added a **partition filter** so a report scanned one day instead of the whole history, and **clustered** the table on the id the query filtered by. For a heavy distinct count I switched to `APPROX_COUNT_DISTINCT`, and I materialized one aggregation that many dashboards repeated. Bytes billed dropped sharply, and so did the bill.

15. Have you implemented partitioning and clustering? Why did you choose those keys?

Partition by event date, because every query is time-bounded, so date pruning removes almost all the data up front. **Cluster by the entity id** (the account or user the query filters and groups by), because it is high-cardinality and appears in nearly every `WHERE` and `GROUP BY`, so block pruning reads only that entity's data. The principle: partition on the dimension every query filters, cluster on the next most selective one.

16. How did you monitor BigQuery costs in production?

- `INFORMATION_SCHEMA.JOBS`: rank queries by bytes billed and slot time, broken down by user and project.
- **Billing export to BigQuery** plus a dashboard for spend over time.
- **Guardrails**: `maximum_bytes_billed` on queries, per-user and per-project quotas, and budget alerts.
- **Capacity pricing** (slots and reservations) when volume is predictable, to cap the bill.
- A **dry run in CI** to catch a query that would scan too much before it ships.

17. Tell me about a difficult BigQuery performance issue you investigated.

A daily report suddenly scanned the whole table instead of one partition. The cause was a filter that **wrapped the partition column in a function**, which defeated partition pruning, so BigQuery could not tell which partitions to skip. The query plan made it obvious: one stage was reading the full table. The fix was to filter the partition column directly, as a plain range on the raw column, which restored pruning and cut the scan back to a single day.

Vendasta-specific questions

1. Many partners run dashboards at the same time. How do slots, reservations, and concurrency limits shape each partner's experience, and how do you stop one heavy tenant from starving the rest?

So let me be honest about where we actually are first: our BigQuery-backed dashboards just run **on-demand at INTERACTIVE priority**, and we have not set up slot reservations in the code. The fairness we do have sits at the application layer, not in BigQuery. Our heavy backfills, for instance, run as Temporal workflows that are singletons per scope, so only one query fires from that path at a time, and if it waits too long on slots it times out and retries with backoff. So really, today a partner's dashboard query is competing in one shared on-demand pool.

On-demand fair scheduling v/s dedicated reservations.

The way on-demand works is every query gets a share of one big slot pool, and BigQuery's fair scheduler splits slots across whatever is running. It is simple and it stretches, but under load a heavy query and a tiny one are waiting in the same pool. Reservations are the other option: you buy a fixed number of slots you control, so you can guarantee capacity and wall off workloads. We are on on-demand today; reservations are the lever I would reach for the moment a heavy tenant started hurting everyone else.

Assigning reservations per workload, and query queuing.

If we did go to reservations, the clean move is separate reservations per workload, say one for interactive dashboards and one for backfills, and assign projects to each, so a batch job physically cannot eat the dashboard slots. And within a reservation, if you run out of slots the queries just queue instead of failing. That is really the answer to one tenant starving the rest: give the dashboards their own guaranteed slots.

Interactive v/s batch query priority.

Interactive queries run as soon as slots free up, and they count against your concurrency limit. Batch queries wait for idle slots and do not count against that limit, the trade being there is no promise of when they start. Our dashboard queries are interactive because a user is sitting there waiting; the thing I would push is to run the heavy, non-urgent backfills at batch priority so they never compete with a live dashboard.

> **HONEST NOTE**
> I should be clear that reservations and slot commitments are not in our application code. If they exist, they live in Terraform, which I cannot see in these repos. Today the thing actually protecting a partner from a heavy tenant is app-layer, the Temporal singletons and timeouts, not a BigQuery reservation.

2. Partner dashboards want sub-second reads, but BigQuery is not a low-latency serving store. How do you bridge that gap?

Here is the thing that surprises people: our low-latency reads do not come from BigQuery at all. The single-post stat badge is served from a Datastore or VStore cache that we refresh asynchronously from the network API, and the Post Performance dashboard is served by **tesseract**, our own SQL query service over VStore, not BigQuery. BigQuery for us is the analytical mirror; it powers a small set of cross-product metrics plus reporting and backfills, where a few seconds of latency is totally fine.

So let me run through the usual ways you bridge that gap, and where each one runs out:

Pre-aggregated rollup tables on a schedule v/s materialized views.

A scheduled rollup table is just a query you run on a schedule that writes a small summary table the dashboard reads, so you control the shape and it is cheap to read, but the data is only as fresh as the last run. A materialized view is one BigQuery keeps fresh for you automatically, but it only supports a limited slice of SQL. Both shrink the scan a lot, but neither one gets you down to single-digit milliseconds.

BI Engine: what it does and where it stops helping.

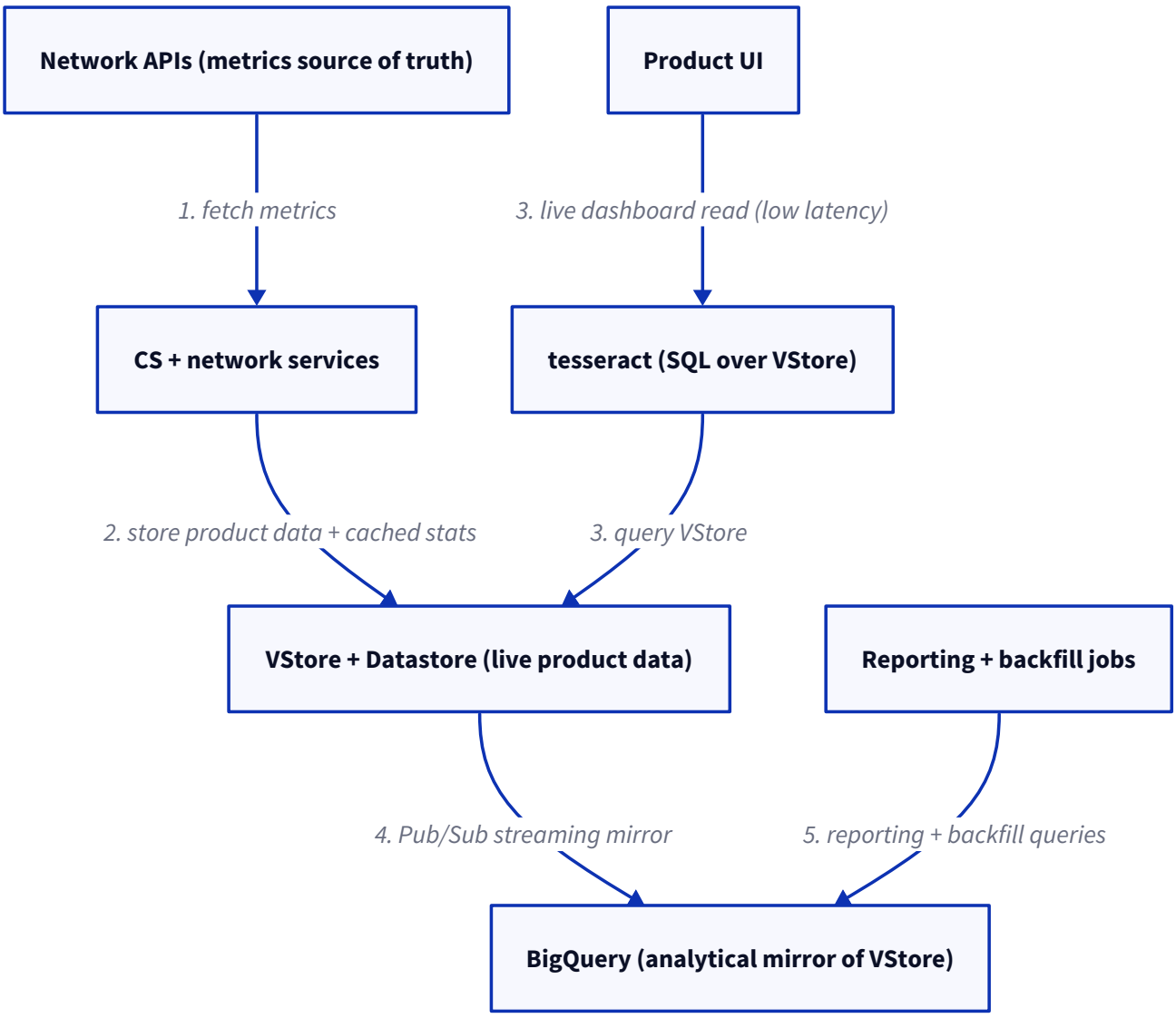
BI Engine is an in-memory layer that caches the hot data and speeds up repeated dashboard queries, and it can get you well under a second for the queries that fit in it. Where it stops helping is when the working set does not fit the memory you reserved, or the queries are high-cardinality and wide and ad-hoc, and it is still not a point-lookup store for a single row.

Exporting aggregates into a serving store (Bigtable, Datastore, Redis): the read-model split.

When you genuinely need sub-second per-entity reads, the pattern is to compute the aggregates in BigQuery and push them out into a serving store: Bigtable for high-throughput key lookups, Datastore or Firestore for document reads, Redis for a hot cache. Then the dashboard just reads the pre-computed answer by key. That is the read-model split, BigQuery on the compute side and the serving store on the read side. What is interesting is that we effectively do the serving side with VStore, tesseract, and Datastore caches instead of a BigQuery-to-store export, so there is no such export pipeline in our code.

3. The product's live data sits in VStore and the networks' own APIs are the source of truth for metrics. Where does BigQuery belong in that picture, and where would using it be the wrong choice?

The way I think about it, BigQuery is our **analytical mirror of VStore**; it is not a serving store and not a source of truth. Every VStore write gets streamed into BigQuery through Pub/Sub, into an append-only table with a dedupe view that keeps the latest version of each row. Then reporting jobs, backfills, and a small set of cross-product dashboard metrics query that mirror, where seconds of latency and slightly-behind freshness are acceptable. The networks' own APIs stay the source of truth for metrics, VStore and Datastore hold the live product data, and tesseract serves the live in-product reads.



The OLTP v/s OLAP boundary, concretely for this product.

Concretely, OLTP for us is VStore and Datastore: a user opens a post and we read that one record or its cached stat in milliseconds. OLAP is BigQuery: something like "sum the reach across all this partner's posts last quarter" scans a lot and takes seconds. So the line is really about latency and access shape. Single-record reads and writes stay in VStore, and the big aggregations go to BigQuery.

Why not serve the in-product per-post stat straight from BigQuery?

Because for a point read it would be slow and expensive. That per-post badge needs one record back in milliseconds, and BigQuery answers in hundreds of milliseconds to seconds, it bills you for the bytes scanned, and it is carrying the streaming lag from the mirror on top. Serving it from the VStore or Datastore cache is faster, cheaper, and fresher for a single-row read. It is just the OLTP-not-OLAP rule in practice.

Federated queries or BigLake as a middle ground: when?

A federated query lets BigQuery read data that lives somewhere else without loading it in first. We actually use this in one spot: a legacy scheduling query runs over a federated view to work out which social profiles are due for a stats refresh. BigLake extends the same idea to files in object storage with consistent governance. I would reach for either when I want to query external or live-ish data now and then without building a whole load pipeline, and I would accept that the scan is slower because BigQuery does not own the data.

4. The product is white-label with many partners, each scoped to its own businesses. How do you design partner-scoped analytics so one partner can never read another's data, while reporting stays fast?

So the way we do it is **shared tables with a `partner_id` column**, and every analytics query filters by `partner_id`, and the account group. We do not do a dataset per partner, and there are no authorized views or row-level security in the app code, so honestly the isolation today rests on the query always carrying the right `partner_id` filter.

One dataset per partner v/s a shared table with a partner_id column: trade-offs at thousands of tenants.

Dataset-per-partner gives you hard isolation, because you grant IAM per dataset, so a wrong query simply cannot cross tenants. But at thousands of partners that is thousands of datasets and tables to create, migrate, and manage, and cross-partner reporting becomes a pain. The shared table with a `partner_id` is simple, it scales to any number of tenants, and cross-partner reporting is trivial, but now the isolation depends on every query filtering correctly, which is an application-enforced boundary, not a storage one. We went with the shared-table model.

> **THE RISK**
>
> With a shared table, a query that forgets the `partner_id` filter reads every partner's data. It is the same shape as an IDOR from the auth file: the filter, not the storage, is the boundary, so the filter can never be optional.

Authorized views and row-level security: where do they fit?

They move the isolation out of the application and into BigQuery. An authorized view only exposes a partner's own rows and is the only thing that partner's principal can query, so there is no raw table scan to abuse. Row-level security attaches a policy to the table that filters rows by the caller's identity automatically. Either one turns "the app has to remember to filter" into "the storage enforces it", which is the right hardening for a shared multi-tenant table. We do not have these in our code today; they would live in the BigQuery IAM and policy layer.

Would you cluster by partner_id, and what does that buy you?

Yes, I would. Clustering by `partner_id` sorts each partition's data by partner, so a query filtered to one partner only reads that partner's blocks, which is block pruning. It buys you both speed and cost, a partner's report scans a fraction of the table instead of the whole thing. It is not an isolation control, a missing filter still reads everything, but paired with partitioning by date it is the main lever that keeps partner-scoped reporting fast on a shared table.